

Grundbausteine:

SELECT <Attributliste>	(= Schema der Ergebnisrelation; * für alle Attribute)
FROM <Relationenliste>	(Relationen aus denen die Tupel stammen)
WHERE <Bedingungen>;	(Bedingungen an die Daten; Verknüpfung über Schlüsselwort AND)

SELECT:

- Entspricht Projektion π der relationalen Algebra
- Umbenennung: `SELECT Titel AS Filmtitel`
- Arithmetik: `SELECT Länge * 3.1415 AS LängeMalPi`
- Konstanten: `SELECT ‚Herr‘ AS Titel`
- Duplikateliminierung δ : `SELECT DISTINCT Titel`

FROM:

- Entspricht Kreuzprodukt $\times$ der relationalen Algebra (falls mehrere Relationen gewählt)

WHERE:

- Entspricht Selektion σ der relationalen Algebra
- WHERE-Teil der Anfrage ist optional
- Operatoren: =, <>, >, <, <=, >=, LIKE, NOT, ANY, ALL, EXISTS, IN, ...
- Kann Kreuzprodukt des FROM-Teils zum Join machen:

WHERE Person.ID = Mensch.ID (Natürlicher Join $\bowtie$)

WHERE Person.Name = Mensch.Vorname (Theta Join $\bowtie_{\theta}$)

Komplexe Anfragen:

```
SELECT <Attributliste>  
FROM <Relationenliste>  
WHERE <Bedingungen>  
GROUP BY <Gruppierungsattribute>  
HAVING <Bedingungen auf Gruppierungsattribute>  
ORDER BY <Attributliste>;
```

GROUP BY ... HAVING:

- Entspricht Gruppierung γ der relationalen Algebra
- Aggregationsoperatoren für SELECT-Statement:
 - AVG(<Attribut>)
 - COUNT(<Attribut>)
 - SUM(<Attribut>)
- HAVING entspricht einer Selektion nach der Gruppierung

ORDER BY:

- Entspricht Sortierung τ der relationalen Algebra
- Sortiert das Ergebnis der Anfrage entsprechend der Attributliste
- Aufsteigend: ORDER BY Vorname, Nachname ASC
- Absteigend: ORDER BY Vorname, Nachname DESC

Datendefinition: Data Definition Language (DDL)

CREATE TABLE <Tabellenname>(<Attributliste mit Datentypen>);

- Aufgabe: neue Tabelle erstellen
- Datentypen mit Länge n bzw. m: CHAR(n), VARCHAR(n), BIT(n), DECIMAL(n,m)
- Datentypen mit impliziter Länge: INT, FLOAT
- Datentypen für Objekte: CLOB, BLOB
- Datentypen für Zeiten: TIME, DATE, TIMESTAMP
- Bsp.: CREATE TABLE Schauspieler (
 - Name CHAR(30),
 - Adresse VARCHAR(255),
 - Geschlecht CHAR(1),
 - Geburtstag DATE);
- Nebenbedingungen:
 - Primärschlüssel: PRIMARY KEY
 - Eindeutigkeit: UNIQUE
 - Default-Werte: DEFAULT <Defaultwert>
 - Nicht-Null: NOT NULL
 - Fremdschlüssel: FOREIGN KEY (<Attributliste>) REFERENCES <Tabellenname>(<Attributliste>)
 - Weitere: CHECK, CREATE TRIGGER, CREATE ASSERTION, ...
- Bsp.: CREATE TABLE Schauspieler (
 - SchauspielerNummer INT PRIMARY KEY,
 - Name CHAR(30) NOT NULL,
 - Adresse VARCHAR(255) NOT NULL UNIQUE,
 - Geschlecht CHAR(1),
 - Geburtstag DATE DEFAULT DATE '0000-00-00',
 - FOREIGN KEY (Adresse) REFERENCES Haus(Adresse));

DROP TABLE <Tabellenname>;

- Aufgabe: bestehende Tabelle löschen
- Bsp.: DROP TABLE Schauspieler;

ALTER TABLE <Tabellenname> <Aktion>;

- Aufgabe: bestehende Tabelle ändern
- ADD: - Attribut hinzufügen
 - Bsp.: ALTER TABLE Schauspieler ADD Telefon CHAR(16)
- DROP: - Attribut löschen
 - Bsp.: ALTER TABLE Schauspieler DROP Geburtstag

CREATE INDEX <Indexname> **ON** <Tabellenname>(<Attributliste des neuen Index>);

- Aufgabe: Index erstellen

DROP INDEX <Indexname>;

- Aufgabe: Index löschen

Datenbearbeitung: Data Modelling Language (DML)

INSERT INTO <Tabellenname>(<Attributliste>) **VALUES** (<Attributliste>);

- Aufgabe: Tupel einfügen

- Bsp.: `INSERT INTO Studio(Name, Nummer) VALUES ('Pixa', 34);`
- Ergebnis einer Anfrage für Einfügen nutzen:
 Bsp.: `INSERT INTO Studios(Name)
 SELECT DISTINCT StudioName
 FROM Film
 WHERE StudioName NOT IN
 (SELECT Name
 FROM Studios);`

DELETE FROM <Tabellenname> **WHERE** <Bedingung>;

- Aufgabe: Tupel löschen
- Bsp.: `DELETE FROM Studio WHERE Name='Pixa';`

UPDATE <Tabellenname> **SET** <Zuweisung> **WHERE** <Bedingung>;

- Aufgabe: Attributwerte ändern
- Bsp.: `UPDATE Studio SET Name='Pixa' WHERE Name='Pi';`

Bulk insert: `IMPORT, LOAD, ...` (à DBMS spezifisch)

Mengenoperationen

(<Anfrage>) **UNION** (<Anfrage>) (Liefert Vereinigung „ $\cup$ “ der beiden Ergebnismengen)

(<Anfrage>) **EXCEPT** (<Anfrage>) (Liefert Differenz „ $-$ “ der beiden Ergebnismengen)

(<Anfrage>) **INTERSECT** (<Anfrage>) (Liefert Schnittmenge „ $\cap$ “ der beiden Ergebnismengen)

- UNION, EXCEPT und INTERSECT nutzen Mengensemantik (à eliminieren Duplikate)
- UNION ALL, EXCEPT ALL und INTERSECT ALL nutzen Multimengensemantik (à erhalten Duplikate)

Join-Varianten

1.) Kreuzprodukt mit Bedingung:

```
SELECT *
FROM <Join-Relation1>, <Join-Relation2>
WHERE <Join-Attribut1> = <Join-Attribut2>;
```

2.) Schlüsselwort:

```
<Tabellenname> CROSS JOIN <Tabellenname>
<Tabellenname> NATURAL JOIN <Tabellenname>
<Tabellenname> NATURAL INNER JOIN <Tabellenname>
<Tabellenname> NATURAL LEFT OUTER JOIN <Tabellenname>
<Tabellenname> NATURAL RIGHT OUTER JOIN <Tabellenname>
<Tabellenname> NATURAL FULL OUTER JOIN <Tabellenname>
```

Sichten

CREATE VIEW <Sichtname> **AS** <Anfrage>;

- Aufgabe: Erstelle eine Sicht für die gegebene SQL-Anfrage